HydroGrid

Smart Water & Electricity Intelligence Platform

BACKEND VIVA PREPARATION GUIDE

Node.js • Express.js • PostgreSQL • JWT • AI/ML • WebSocket

Full Stack Developer Project | April 2026

What this guide covers:

- Node.js event loop, modules, non-blocking I/O
- Express.js middleware chain, routing, error handling
- PostgreSQL (Supabase) — connection pool, parameterized queries
- JWT authentication, bcrypt, Google OAuth flow
- AI/ML: Z-score anomaly detection & exponential smoothing
- WebSocket live feed, Groq LLM integration
- 60+ detailed Q&A for every backend concept

SECTION 1 — PROJECT OVERVIEW & BACKEND SCOPE

HydroGrid is a production-level Full-Stack SaaS application for monitoring, analyzing, and optimizing water and electricity consumption. The backend is a RESTful API built with Node.js and Express.js, backed by PostgreSQL (hosted on Supabase). It handles authentication, usage data, alerts, reports, admin operations, AI/ML analytics, real-time WebSocket streaming, and email notifications.

Backend File Structure

```
server/
  server.js              !• Entry point: Express app + WebSocket server
  package.json           !• Dependencies & scripts
  config/
    db.js                !• pg connection pool + connectDB()
  middleware/
    auth.js              !• protect (JWT) + adminOnly (role check)
    errorHandler.js      !• Global error response normalizer
  routes/
    auth.js              !• POST register, login, google; GET/PUT profile
    usage.js             !• CRUD usage data, dashboard, leaderboard
    alerts.js            !• GET, mark-read, delete alerts
    reports.js           !• PDF/CSV export
    admin.js             !• Stats, users list (admin only)
    ai.js                !• Anomaly, forecast, recommendations, chat
    ml.js                !• Training data, state analysis
  controllers/
    authController.js    !• register, login, google OAuth, profile
    usageController.js   !• addUsage, getDashboardStats, leaderboard
    alertController.js   !• getAlerts, markRead, deleteAlert
    adminController.js   !• getStats, getUsers, getOverview
    mlController.js      !• getTrainingData, trainModel, states analysis
    reportController.js  !• generatePDF, exportCSV
  utils/
    aiEngine.js          !• Z-score + Exponential Smoothing algorithms
    groqAI.js            !• Groq LLaMA 3-8B wrapper (singleton)
    emailService.js      !• Nodemailer: welcome + alert emails
    seedData.js          !• Database seeder for development
```

SECTION 2 — NODE.JS FUNDAMENTALS

What is Node.js?

Node.js is a JavaScript runtime built on Chrome's V8 engine. It executes JavaScript outside the browser, enabling server-side scripting. Its key differentiator is the non-blocking, event-driven I/O model — it can handle thousands of concurrent connections without creating a thread per request.

The Event Loop (Critical Concept)

```
Call Stack          !’ Executes synchronous JS code
Node APIs           !’ Offloads async ops (fs, network, timers)
Callback Queue      !’ Completed callbacks waiting to run
Microtask Queue     !’ Promises / async-await (higher priority)

Loop phases (simplified):
  1. Timers          !’ setTimeout / setInterval callbacks
  2. I/O Callbacks   !’ Network, file system callbacks
  3. Poll            !’ Retrieve new I/O events
  4. Check           !’ setImmediate callbacks
  5. Close Callbacks !’ e.g., socket.on("close")
```

Q: What is non-blocking I/O? CORE

A: Instead of waiting for DB/file/network operations to complete, Node.js delegates them to the OS and continues executing other code. When the operation finishes, the callback is placed in the event queue and executed. This allows a single thread to handle thousands of concurrent requests.

Q: What is the difference between async/await and callbacks? CORE

A: Callbacks are the old way — they lead to "callback hell" (deeply nested code). Promises flattened this with .then() chains. async/await is syntactic sugar over Promises — it lets you write async code that reads like synchronous code, making it easier to read and handle errors with try/catch.

Q: What is require() in Node.js? CORE

A: Node.js uses CommonJS modules. require("module") loads and caches a module. The first call loads it; subsequent calls return the cached export. ES Modules (import/export) are also supported but require .mjs extension or "type":"module" in package.json. This project uses CommonJS (require).

Q: What is process.env? CORE

A: An object that exposes environment variables. dotenv.config() loads .env file into process.env at startup. Used for secrets like JWT_SECRET, DATABASE_URL, GROQ_API_KEY — never hardcoded.

SECTION 3 — EXPRESS.JS DEEP DIVE

What is Express.js?

Express.js is a minimal, unopinionated web framework for Node.js. It provides routing, middleware composition, and HTTP utilities. Every incoming request flows through a chain of middleware functions (the "pipeline") until a response is sent.

Express Middleware Chain in server.js

```
Incoming HTTP Request
        %
        %¼
   cors()              !’ Sets CORS headers, rejects unknown origins
        %
        %¼
   express.json()      !’ Parses JSON body into req.body (10MB limit)
        %
        %¼
   express.urlencoded()!’ Parses form data
        %
        %¼
   Route Handler       !’ /api/auth, /api/usage, /api/ai, etc.
        %
        %¼
   protect middleware  !’ JWT verification (on protected routes)
        %
        %¼
   Controller function !’ Business logic + DB query
        %
        %¼
   res.json()          !’ Send JSON response
        %
        %¼ (on error)
   errorHandler        !’ Normalized error response
```

Q: What is Express middleware? KEY

A: A function with signature (req, res, next). It can read/modify req/res, execute code, or call next() to pass to the next middleware. If next() is not called, the request hangs. Error-handling middleware has 4 params: (err, req, res, next).

Q: What is app.use() vs app.get()? KEY

A: app.use(path, fn) matches any HTTP method and is used for middleware/routers. app.get(path, fn) only matches HTTP GET. app.use("/api/auth", authRoutes) mounts the auth router at that prefix.

Q: What is an Express Router? KEY

A: express.Router() creates a mini-application with its own middleware and routes. Allows route logic to be split into separate files. The main app mounts routers with app.use(). This keeps server.js clean.

Q: What does next(error) do? KEY

A: Calling next(err) skips all remaining middleware and jumps directly to the error-handling middleware (identified by 4 parameters). Used in try/catch blocks: catch(error) { next(error) }.

Q: How is 404 handled? KEY

A: After all routes are defined, a catch-all middleware with no path matches any unmatched route and returns 404. It must be defined AFTER all other routes.

CORS Configuration

```
app.use(cors({
  origin: function(origin, callback) {
    // Allow: no origin (curl/mobile), localhost:*, *.vercel.app, CLIENT_URL
    const allowed = [process.env.CLIENT_URL, "http://localhost:5173", ...]
    if (!origin) return callback(null, true);
    if (allowed.includes(origin) || origin.endsWith(".vercel.app"))
      return callback(null, true);
    callback(new Error("Not allowed by CORS"));
  },
  credentials: true   // Allows cookies/auth headers cross-origin
}));
```

Q: What is CORS and why is it needed?

KEY

A: Cross-Origin Resource Sharing is a browser security mechanism. It blocks requests from http://localhost:5173 to http://localhost:5000 by default (different port = different origin). The server must send Access-Control-Allow-Origin header to permit the frontend. The cors() middleware handles this automatically.

Q: What does credentials: true mean in CORS?

KEY

A: It allows the browser to include cookies and Authorization headers in cross-origin requests. Without it, the Bearer token would not be sent.

SECTION 4 — DATABASE: POSTGRESQL & SUPABASE

Connection Pool (config/db.js)

```
const { Pool } = require("pg");

// Parse DATABASE_URL environment variable
const pool = new Pool({
  user, password, host, port, database,
  ssl: { rejectUnauthorized: false }  // Required for Supabase
});

// Test connection on startup
const connectDB = async () => {
  const client = await pool.connect();
  console.log("'  Supabase PostgreSQL Connected");
  client.release();  // Return connection to pool
};

// Utility wrapper
module.exports = {
  pool,
  connectDB,
  query: (text, params) => pool.query(text, params)
};
```

Q: What is a PostgreSQL connection pool? DB

A: pg.Pool maintains a set of pre-established DB connections. Instead of creating/destroying a connection per query (expensive — TCP handshake + auth), the pool reuses connections. Default pool size is 10 connections. client.release() returns a connection to the pool after use.

Q: What is Supabase? DB

A: An open-source Firebase alternative built on PostgreSQL. Provides a managed, hosted PostgreSQL database with REST, real-time, and auth APIs. We connect to it using the pg driver with a connection string from environment variables. SSL is required for secure connections.

Q: What is SSL in the database connection? DB

A: SSL encrypts the data transmitted between Node.js server and Supabase PostgreSQL. rejectUnauthorized: false accepts self-signed certificates (common in hosted services like Supabase).

Q: Why use parameterized queries ($1, $2)? DB

A: Prevents SQL Injection. User input is passed as parameters, never concatenated into the SQL string. The pg driver sends the query and parameters separately — the DB treats parameters as data, never as SQL code. Example: "SELECT * FROM users WHERE email = $1" with params [email] — malicious SQL in email is harmless.

Q: What SQL tables does this project use? DB

A: Three main tables: (1) users — stores account data, role, state, settings; (2) usage_data — water/electricity readings per user with timestamp; (3) alerts — threshold-triggered notifications per user. Relationships use foreign keys (user_id).

Database Schema

users

id, name, email, password, role, state,
settings, badges, created_at

role: user|admin; settings JSONB

usage_data

id, user_id (FK), type, value, unit,
timestamp, state

type: water|electricity

alerts

id, user_id (FK), type, severity, message,
read, timestamp

severity: green|yellow|red

Example Queries Used in Code

```
-- Duplicate email check
SELECT id FROM users WHERE email = $1

-- Insert new user
INSERT INTO users (name, email, password, state)
  VALUES ($1, $2, $3, $4) RETURNING *

-- Aggregate usage by day
SELECT date_trunc('day', timestamp) as date,
       type, SUM(value) as value
  FROM usage_data
 WHERE user_id = $1 AND timestamp >= $2
 GROUP BY date, type
 ORDER BY date ASC

-- Leaderboard (efficiency ranking)
SELECT u.name, u.state,
       AVG(d.value) as avg_usage
  FROM users u
  JOIN usage_data d ON u.id = d.user_id
 GROUP BY u.id, u.name, u.state
 ORDER BY avg_usage ASC LIMIT 10
```

SECTION 5 — AUTHENTICATION SYSTEM

JWT Flow Diagram

```
CLIENT                                    SERVER
  %                                          %
  % % %  POST /api/auth/login % % % % % % % % % % % % % % % % %º %
  %    { email, password }                   %
  %                                          %  1. Fetch user by email (SQL)
  %                                          %  2. bcrypt.compare(pass, hash)
  %                                          %  3. jwt.sign({ id }, SECRET, { expiresIn: "30d" })
  % %Ä%  { success, token, user } % % % % % % % % % % %  %
  %
  %   (store token in localStorage)
  %
  % % %  GET /api/usage/dashboard % % % % % % % % % % %º %
  %    Authorization: Bearer <token>         %
  %                                          %  protect middleware:
  %                                          %    1. Extract token from header
  %                                          %    2. jwt.verify(token, SECRET)
  %                                          %    3. SELECT user WHERE id = decoded.id
  %                                          %    4. req.user = user
  %                                          %    5. next()  !’ controller runs
  % %Ä%  { success, data } % % % % % % % % % % % % % % % % % %  %
```

bcrypt Password Hashing

```
// Registration: hash before storing
const salt        = await bcrypt.genSalt(10);   // 2^10 = 1024 rounds
const hashedPass  = await bcrypt.hash(password, salt);
// Stored in DB: $2b$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LWFZwGS.K6T

// Login: verify without knowing original password
const isMatch = await bcrypt.compare(plainPassword, storedHash);
// bcrypt extracts the salt from the hash and re-hashes the input
```

JWT Structure

```
JWT = Base64(header) . Base64(payload) . Signature

Header  : { "alg": "HS256", "typ": "JWT" }
Payload : { "id": "user-uuid", "iat": 1714300000, "exp": 1716892000 }
Signature: HMACSHA256(header + "." + payload, JWT_SECRET)

// Generation
jwt.sign({ id: user.id }, process.env.JWT_SECRET, { expiresIn: "30d" })

// Verification (in protect middleware)
const decoded = jwt.verify(token, process.env.JWT_SECRET);
// decoded.id used to fetch user from DB
```

Google OAuth Flow

```
1. User clicks "Sign in with Google" on frontend
2. Google redirects with id_token to POST /api/auth/google
```

3. Server: googleClient.verifyIdToken({ idToken, audience: CLIENT_ID })

4. Extract { name, email, picture } from Google payload

5. Check if user exists: SELECT * FROM users WHERE email = $1

6. If not: INSERT new user (no password — Google verified)

7. Generate HydroGrid JWT token

8. Return { token, user } !’ frontend stores and uses it like normal JWT

Q: Why JWT over sessions?

A: JWT is stateless — no server-side session store needed. The token contains the user's identity, signed with a secret. Any server instance can verify it independently. This scales horizontally — no shared session DB needed. Sessions require a central store (Redis, DB) for multi-server deployments.

Q: Why is bcrypt cost factor 10? AUTH

A: Cost factor 10 means 2^10 = 1024 hashing rounds. This takes ~100ms on a modern CPU — fast enough for users, but too slow for brute-force attacks (10,000 attempts would take ~17 minutes). Increasing to 12 doubles the time.

Q: What is the difference between authentication and authorization? AUTH

A: Authentication: verifying WHO you are (login, JWT verify). Authorization: verifying WHAT you can do (protect checks if logged in; adminOnly checks if you have admin role). These are two separate middleware functions.

Q: What happens when JWT expires? AUTH

A: jwt.verify() throws TokenExpiredError. The protect middleware returns 401. The frontend Axios response interceptor catches 401, clears localStorage (removes token and user data), and redirects to /login automatically.

Q: What is the purpose of the salt in bcrypt? AUTH

A: A salt is a random string added before hashing. It ensures two identical passwords produce different hashes, defeating rainbow table attacks (pre-computed hash lookups). bcrypt stores the salt inside the hash string, so no separate storage is needed.

SECTION 6 — MIDDLEWARE IN DETAIL

protect Middleware (middleware/auth.js)

```
const protect = async (req, res, next) => {
  let token;
  // 1. Extract token from "Authorization: Bearer <token>"
  if (req.headers.authorization?.startsWith("Bearer"))
    token = req.headers.authorization.split(" ")[1];

  if (!token)
    return res.status(401).json({ success:false, message:"No token" });

  // 2. Verify token signature & expiry
  const decoded = jwt.verify(token, process.env.JWT_SECRET);

  // 3. Fetch fresh user data from DB (in case role/email changed)
  const result = await query(
    "SELECT id, name, email, role, state FROM users WHERE id = $1",
    [decoded.id]
  );

  if (!result.rows[0])
    return res.status(401).json({ success:false, message:"User not found" });

  req.user = result.rows[0];  // Attach user to request
  next();                     // Pass to controller
};
```

adminOnly Middleware

```
const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === "admin")
    return next();
  return res.status(403).json({ message: "Access denied - admin only" });
};

// Usage on admin routes:
router.use(protect, adminOnly);  // Both middleware applied
```

Global Error Handler (middleware/errorHandler.js)

```
const errorHandler = (err, req, res, next) => {
  let statusCode = err.statusCode || 500;
  let message    = err.message    || "Internal Server Error";

  if (err.name === "JsonWebTokenError")  { statusCode = 401; message = "Invalid token"; }
  if (err.name === "TokenExpiredError")  { statusCode = 401; message = "Token expired"; }

  res.status(statusCode).json({
    success: false,
    message,
    // Stack trace only in development (security: no info leakage in prod)
    ...(process.env.NODE_ENV === "development" && { stack: err.stack })
  });
};
```

Q: Why does errorHandler have 4 parameters?

A: Express identifies error-handling middleware by the 4-parameter signature (err, req, res, next). When next(err) is called from any route or middleware, Express skips all regular middleware and jumps directly to the first error handler.

Q: Why fetch user from DB again in protect middleware instead of just using JWT payload? MW

A: The JWT payload is only generated at login. If a user is deleted, banned, or their role is changed AFTER they received a token, the JWT still looks valid. Re-fetching from DB ensures the user still exists and has the current role.

Q: What is the difference between 401 and 403? MW

A: 401 Unauthorized: you are not authenticated (no token, invalid token, expired token). 403 Forbidden: you ARE authenticated but lack permission (valid JWT, but not admin role). adminOnly returns 403, not 401.

SECTION 7 — CONTROLLERS & BUSINESS LOGIC

authController.js — Key Functions

Function	Method	Key Logic
register()	POST /api/auth/register	Validate !' check duplicate !' bcrypt hash !'
login()	POST /api/auth/login	Fetch user !' bcrypt.compare !' JWT
googleAuth()	POST /api/auth/google	Verify Google ID token !' upsert user !' JWT
getProfile()	GET /api/auth/profile	SELECT * WHERE id = req.user.id
updateProfile()	PUT /api/auth/profile	UPDATE users SET name=$1,avatar=$2,settings=$3 WHERE id=$4

usageController.js — Key Functions

Function	Route	Description
addUsage()	POST /api/usage	INSERT reading, auto-create alert if
getUsage()	GET /api/usage	SELECT with optional type/date filters,
getDashboardStats()	GET /api/usage/dashboard	Aggregated totals, averages, trends for
simulateIoT()	POST /api/usage/simulate	Inserts random readings to simulate smart
getLeaderboard()	GET /api/usage/leaderboard	JOIN users+usage GROUP BY user,
getCarbonFootprint()	GET /api/usage/carbon	electricity kWh × 0.82 kgCO2/kWh + water
getTariffEstimate()	GET /api/usage/tariff-estimate	Slab-based cost calc from INDIA_TARIFFS
getMapStateStats()	GET /api/usage/map	AVG usage per state for India map choropleth

Threshold Alert Logic (in addUsage)

```
// After inserting a reading, check if threshold is exceeded
const WATER_LIMIT       = 500;   // litres/day
const ELECTRICITY_LIMIT = 20;    // kWh/day

if (type === "water" && value > WATER_LIMIT) {
  await query(`INSERT INTO alerts (user_id, type, severity, message)`,
    [userId, "water", "red",
     `High water usage: ${value}L exceeds daily limit of ${WATER_LIMIT}L`]
  );
  sendAlertEmail(user, alert).catch(console.error);  // Non-blocking email
}
```

Q: What is IoT simulation in addUsage? CTRL

A: simulateIoT() generates random realistic water (50–800L) and electricity (5–35kWh) readings using Math.random() for the past 30 days. This populates the DB for demo purposes, simulating what a smart meter would send automatically.

Q: How does the leaderboard work? CTRL

A: It JOINs users with usage_data, groups by user, computes average consumption per day, and orders ascending (lower = more efficient). The most efficient consumers rank highest. State filtering is supported.

Q: How is carbon footprint calculated? CTRL

A: Electricity: kWh × 0.82 kg CO , (India's grid emission factor). Water: litres × 0.001 kg CO , (pumping/ treatment energy). Returns total CO , in kg and the equivalent number of trees needed to offset it.

SECTION 8 — AI & ML ENGINE

1. Anomaly Detection — Z-Score Algorithm (aiEngine.js)

```
function detectAnomalies(usageData, threshold = 2.5) {
  const values = usageData.map(d => d.value);

  // Step 1: Calculate mean (;Â’  æB 7F æF &B FWf– F–öâ ƒÃ)
  const n    = values.length;
  const mean = values.reduce((a, b) => a + b) / n;
  const std  = Math.sqrt(values.reduce((sq, v) => sq + (v - mean)**2, 0) / n);

  // Step 2: Compute Z-score for each data point
  return usageData
    .map(r => ({ ...r, zscore: (r.value - mean) / std }))
    .filter(r => Math.abs(r.zscore) > threshold)  // |z| > 2.5
    .map(r => ({
      date:      r.date,
      value:     r.value,
      expected:  mean,
      deviation: r.value - mean,
      reason:    r.value > mean ? "SPIKE" : "DROP",
      severity:  Math.abs(r.zscore)   // Higher = more anomalous
    }));
}
```

A Z-score of 2.5 means the value is 2.5 standard deviations from the mean. Under a normal distribution, this covers 98.8% of expected values — so only genuinely unusual readings (top 1.2%) are flagged as anomalies.

2. 30-Day Forecasting — Exponential Smoothing

```
function forecastUsage(usageData, days = 30, alpha = 0.3) {
  const values = usageData.map(d => d.value);
  let level = values[0];

  // Step 1: Fit model to historical data
  //   Formula: S_t = ;  r …÷B ² ƒ  Ò ±) × S_(t-1)
  for (let i = 1; i < values.length; i++)
    level = alpha * values[i] + (1 - alpha) * level;

  // Step 2: Generate future predictions with seasonal adjustment
  for (let i = 1; i <= days; i++) {
    const forecastDate = new Date(lastDate);
    forecastDate.setDate(forecastDate.getDate() + i);

    // Use historical same-day-of-week average for seasonality
    const dayOfWeek    = forecastDate.getDay();  // 0=Sun, 6=Sat
    const sameDay      = usageData.filter(d => new Date(d.date).getDay() === dayOfWeek);
    const dayAvg       = sameDay.reduce((a, b) => a + b.value, 0) / sameDay.length;

    const variance = 0.1 * level;   // ±10% confidence interval
    forecast.push({
      date:       forecastDate.toISOString(),
      predicted:  Math.round(dayAvg),
      lower:      Math.round(dayAvg - variance),
      upper:      Math.round(dayAvg + variance),
      confidence: 0.85
    });
```

```
}
```

```
    return forecast;
```

```
}
```

3. Groq AI — LLaMA 3-8B Integration (groqAI.js)

```
class GroqAI {
  constructor() {
    this.client  = new Groq({ apiKey: process.env.GROQ_API_KEY });
    this.enabled = !!process.env.GROQ_API_KEY;
  }

  async generateJSONResponse(prompt, schemaExample) {
    const message = await this.client.messages.create({
      model:       "llama3-8b-8192",  // 8B params, 8192 token context
      max_tokens:  2000,
      temperature: 0.2,              // Near-deterministic output
      messages: [
        { role: "system", content: "Return ONLY valid JSON, no markdown" },
        { role: "user",   content: prompt + "\n" + schemaExample }
      ]
    });
    return this._extractJSON(message.content[0].text);
  }
}

// Singleton pattern — one instance shared across all AI routes
let groqInstance = null;
function getGroqAI() {
  if (!groqInstance) groqInstance = new GroqAI();
  return groqInstance;
}
```

4. AI Route Caching

```
// In-memory cache (server lifetime) for Groq API responses
const aiMemoryCache = {};
const CACHE_TTL = 1000 * 60 * 60 * 24;  // 24 hours in ms

// Cache key: userId_endpointName (scoped per user)
const getCachedResponse = (req, endpoint) => {
  const key = `${req.user.id}_${endpoint}`;
  const hit = aiMemoryCache[key];
  if (hit && (Date.now() - hit.timestamp < CACHE_TTL))
    return hit.data;
  return null;
};
```

Q: What is a Z-score and when is it used? AI

A: Z = (x " ;Â• / <2à It standardizes a value relative to the distribution. |Z| > 2.5 means the value is in the extreme 1.2% of the distribution. Used here to detect abnormal utility consumption spikes or drops.

Q: Why alpha = 0.3 in exponential smoothing? AI

A: ; controls how reactive the model is. ; = 0.3 means 30% weight on the latest observation and 70% on the historical trend. Low ; produces smoother, more conservative forecasts — appropriate for utility usage which doesn't change drastically day to day.

Q: What is the Singleton pattern in GroqAI? AI

A: Only one GroqAI instance is created (lazy initialization). getGroqAI() returns the existing instance if it exists. This avoids creating a new Groq HTTP client on every request, saving memory and connection

overhead.

Q: What is temperature = 0.2 in the LLM call? AI

A: Temperature controls randomness in LLM output. 0 = fully deterministic, 1 = maximum creativity. 0.2 is near-deterministic — ideal for structured JSON responses where we need consistent, reliable output rather than creative variation.

Q: Why is there a 24-hour cache for AI responses? AI

A: Groq API calls cost money (tokens) and have rate limits. Usage patterns don't change dramatically within 24 hours. Caching reuses the same analysis response, reducing API calls, cost, and latency.

Q: What is the difference between statistical ML (aiEngine.js) and LLM (groqAI.js)? AI

A: aiEngine.js uses classical statistical algorithms (Z-score, exponential smoothing) implemented from scratch in JS — deterministic, fast, no API key needed. groqAI.js calls a cloud LLM (LLaMA 3) for natural language insights, which requires an API key and external network call.

SECTION 9 — WEBSOCKET LIVE FEED

```
// Attach WebSocket server to the same HTTP server instance
const wss = new WebSocketServer({ server, path: "/ws/live" });

wss.on("connection", (ws) => {
  app.locals.wsClientCount = wss.clients.size;

  // Send immediate confirmation
  ws.send(JSON.stringify({
    type: "connected",
    message: "HydroGrid live feed connected",
    timestamp: new Date().toISOString()
  }));

  ws.on("close", () => { app.locals.wsClientCount = wss.clients.size; });
});

// Broadcast live IoT metrics every 10 seconds to ALL connected clients
setInterval(() => {
  const payload = {
    type:          "live_metrics",
    waterLpm:      parseFloat((18 + Math.random() * 6).toFixed(2)),  // L/min
    electricityKw: parseFloat((1.2 + Math.random() * 2.5).toFixed(2)), // kW
    alerts:        Math.random() > 0.9 ? 1 : 0,   // 10% chance of alert
    timestamp:     new Date().toISOString()
  };
  wss.clients.forEach(client => {
    if (client.readyState === 1)   // 1 = OPEN
      client.send(JSON.stringify(payload));
  });
}, 10000);
```

Q: What is WebSocket? WS

A: A persistent, full-duplex communication protocol. Unlike HTTP (request-response), WebSocket opens a single TCP connection and keeps it open — the server can push data to the client at any time without the client asking. Initiated by an HTTP upgrade handshake.

Q: WebSocket vs HTTP polling? WS

A: Polling: client sends GET every N seconds — wasteful, high latency, scales poorly. WebSocket: server pushes when data is ready — efficient, low latency, single persistent connection per client.

Q: What does client.readyState === 1 check? WS

A: WebSocket states: 0=CONNECTING, 1=OPEN, 2=CLOSING, 3=CLOSED. We only send to OPEN connections to avoid errors when a client is disconnecting.

Q: How does the WebSocket share the same port as HTTP? WS

A: ws.WebSocketServer({ server }) attaches to the existing HTTP server. The WebSocket upgrade handshake is an HTTP request — the same server handles it. Port 5000 serves both REST API and WebSocket on path /ws/live.

SECTION 10 — EMAIL SERVICE (NODEMAILER)

```
const transporter = nodemailer.createTransport({
  host:   process.env.SMTP_HOST || "smtp.gmail.com",
  port:   parseInt(process.env.SMTP_PORT) || 587,
  secure: process.env.SMTP_SECURE === "true",  // false for TLS/STARTTLS
  auth: {
    user: process.env.SMTP_USER,  // Gmail address
    pass: process.env.SMTP_PASS,  // Gmail App Password (not account pass)
  }
});

// Two email types:
// 1. sendWelcomeEmail(user)   !’ fired on registration
// 2. sendAlertEmail(user, alert) !’ fired when threshold exceeded

// Email sending is non-blocking: fire and forget
sendWelcomeEmail(user).catch(e => console.error("Email Error:", e));
```

Q: What is Nodemailer? EMAIL

A: A Node.js library for sending emails via SMTP. Supports Gmail, Mailtrap, SendGrid, etc. createTransport() configures the SMTP connection. sendMail() sends the email. Returns messageId on success.

Q: What is a Gmail App Password? EMAIL

A: Google blocks direct Gmail password use with SMTP for security. An App Password is a 16-character code generated in Google Account Security settings. It allows specific apps (like Nodemailer) to authenticate without exposing your main password.

Q: Why is email sending non-blocking (fire and forget)? EMAIL

A: Email delivery is slow (network call to SMTP server). If we awaited it, the user would wait 1-2 seconds for the API response. Since email failure does not affect registration success, we fire it asynchronously and catch errors separately without blocking the response.

SECTION 11 — INDIA TARIFF SYSTEM

HydroGrid implements India-specific progressive (slab-based) tariff calculations for both electricity and water, covering multiple states. This is the same billing method used by actual Indian utilities (MSEDCL, BSES, BESCOM, etc.).

How Slab-Based Tariff Works

```
// Maharashtra Electricity Slabs:
//   0 – 100  units !’  ¹4.41/unit
//   101 – 300 units !’  ¹8.82/unit
//   301 – 500 units !’  ¹11.72/unit
//   > 500    units !’  ¹12.92/unit

function calculateSlabCost(usage, slabs) {
  let cost = 0, remaining = usage;
  let prevUpto = 0;
  for (const slab of slabs) {
    const slabQty = Math.min(remaining, slab.upto - prevUpto);
    if (slabQty <= 0) break;
    cost      += slabQty * slab.rate;
    remaining -= slabQty;
    prevUpto   = slab.upto;
    if (remaining <= 0) break;
  }
  return cost;
}

// Example: 400 units in Maharashtra
//   First 100 × 4.41 =  ¹441
//   Next  200 × 8.82 =  ¹1,764
//   Next  100 × 11.72 =  ¹1,172
//   Total =  ¹3,377
```

States with Special Rates

State	Special Rule
Delhi	Free 20,000 L water/month for domestic consumers
Tamil Nadu	First 100 electricity units FREE (government subsidy)
Karnataka	Lowest electricity rate: ¹4.15 for first 50 units
Gujarat	Lowest base electricity rate: ¹3.05 for first 50 units
Maharashtra	Highest slab rates (MSEDCL commercial rates)

SECTION 12 — SECURITY (OWASP TOP 10)

OWASP Threat	Risk	Mitigation in HydroGrid
A01: Broken Access Control	Users accessing other users data	All queries use WHERE user_id =
A02: Cryptographic Failures	Plain-text passwords	bcrypt with salt (cost 10) — irreversible
A03: SQL Injection	Malicious SQL via inputs	All queries use $1/$2 parameterized
A05: Security Misconfiguration	CORS too permissive	Whitelist: only CLIENT_URL, localhost:*,
A07: Auth Failures	Brute force, token forgery	bcrypt slows brute force; JWT signed with
A09: Logging/Monitoring	No visibility into errors	console.error on all failures; error stack only
Info Disclosure	Stack traces in prod	stack only sent when NODE_ENV ===
XSS (Cross-site Scripting)	Script injection	React auto-escapes JSX; all API responses
Secret Exposure	Keys in source code	.env file with dotenv; .env in .gitignore;
Request Bombing	Oversized payloads	express.json({ limit: "10mb" }) caps request body size

Q: What is SQL Injection?

SEC

A: An attacker inserts malicious SQL into input fields. Example: email = "admin@x.com' OR '1'='1" — this could bypass login checks. Parameterized queries ($1, $2) prevent this by treating inputs as data, never as SQL.

Q: What is CORS and why is it a security measure?

SEC

A: CORS prevents malicious websites from making requests to your API using a logged-in user's browser cookies/tokens. By whitelisting only known origins, we ensure only our frontend (and not attacker sites) can call the API from a browser.

Q: What is XSS and how does React protect against it?

SEC

A: Cross-Site Scripting: attacker injects JavaScript into your page. React prevents this by escaping all JSX output by default — <script> tags and HTML entities in data are rendered as text, not executed.

SECTION 13 — npm PACKAGES EXPLAINED

Package	Version	What it does	Where used
express	^4.21	Web framework — routing, middleware	server.js (entire app)
pg	^8.20	PostgreSQL client — Pool, queries	config/db.js, all controllers
jsonwebtoken	^9.0	Sign and verify JWT tokens	authController.js, auth.js
bcryptjs	^2.4	Hash and compare passwords	authController.js (register/login)
cors	^2.8	Set Access-Control headers	server.js (global middleware)
dotenv	^16.4	Load .env into process.env	server.js (first line)
ws	^8.14	WebSocket server	server.js (/ws/live endpoint)
nodemailer	^8.0	Send emails via SMTP	utils/emailService.js
groq-sdk	^0.3	Groq API client for LLaMA 3	utils/groqAI.js
pdfkit	^0.15	Generate PDF files	controllers/reportController.js
simple-statistics	^7.8	Statistical functions	Used in ML calculations
google-auth-library	^10.6	Verify Google OAuth ID tokens	authController.js (googleAuth)

Q: Why use bcryptjs instead of the native crypto module? PKG

A: bcryptjs is specifically designed for password hashing — it uses the bcrypt algorithm with salt and work factor. Node.js crypto module provides general cryptography but lacks bcrypt's built-in salt embedding and work factor control. bcrypt is the industry standard for password storage.

Q: What is the difference between pdfkit and jsPDF? PKG

A: pdfkit is a server-side Node.js PDF generator — runs on the backend, produces binary PDF streams. jsPDF is a browser-side JavaScript library. HydroGrid uses pdfkit because reports are generated on the server (can access full DB data) and served as downloads.

SECTION 14 — ENVIRONMENT VARIABLES

```
# .env file (NEVER commit to git)

# Server
PORT=5000
NODE_ENV=development

# Database (Supabase PostgreSQL)
DATABASE_URL=postgresql://user:password@host:5432/dbname

# Authentication
JWT_SECRET=your_super_long_random_secret_key_here

# Google OAuth
GOOGLE_CLIENT_ID=your_google_oauth_client_id.apps.googleusercontent.com

# Groq AI
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxx

# Email (Gmail SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=youremail@gmail.com
SMTP_PASS=your_app_password

# Frontend URL (for CORS whitelist)
CLIENT_URL=http://localhost:5173
```

Q: Why must .env be in .gitignore? ENV

A: The .env file contains secrets: database credentials, JWT secret, API keys. Committing these to git (especially public repos) exposes them to attackers who can access your database, impersonate your server's JWT, and exhaust your API credits.

Q: What happens if JWT_SECRET is weak? ENV

A: JWT signatures use HMAC-SHA256. A weak/short secret is vulnerable to brute-force attacks — an attacker could forge tokens and authenticate as any user. Use a 256-bit (32+ character) random string.

SECTION 15 — DOCKER & DEPLOYMENT

docker-compose.yml Services

```
services:
  hydrogrid-api:            # Node.js backend
    build: ./server
    ports:  ["5000:5000"]
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - JWT_SECRET=${JWT_SECRET}
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/api/health"]
      interval: 30s
      retries: 3
    restart: unless-stopped

  hydrogrid-web:            # Nginx serving React build
    build: ./client
    ports:  ["80:80"]
    depends_on: [hydrogrid-api]
```

Render.com Deployment (render.yaml)

```
# Backend: Web Service (Node.js)
type: web
name: hydrogrid-api
env: node
buildCommand: npm install
startCommand: node server.js
envVars:
  - key: DATABASE_URL
    fromDatabase: { name: hydrogrid-db, property: connectionString }

# Frontend: Static Site (Vite build)
type: static
name: hydrogrid-web
buildCommand: npm run build
staticPublishPath: ./dist
```

Q: What is Docker? DEPLOY

A: A containerization platform. A Docker container packages the app with all its dependencies (Node.js, npm packages) into an isolated, reproducible unit. "Works on my machine" stops being a problem — the container runs identically everywhere.

Q: What is docker-compose? DEPLOY

A: A tool to define and run multi-container Docker apps with a YAML file. One docker compose up command starts all services (API + web), with networking, volumes, and env vars configured.

Q: What is a health check endpoint (/api/health)? DEPLOY

A: A lightweight endpoint that returns 200 OK when the server is running. Docker uses it to determine if the container is healthy. Load balancers and monitoring tools ping it. If it fails, Docker/Render restarts the container.

SECTION 16 — REQUEST LIFECYCLE (END-TO-END)

Understanding the full journey of a request from browser to database and back.

Example: POST /api/usage (Add Usage Reading)

```
BROWSER
  %
  %   POST http://localhost:5000/api/usage
  %   Headers: { Authorization: "Bearer eyJhbGciOi..." }
  %   Body:    { "type": "water", "value": 750, "unit": "litres" }
  %
  %¼
SERVER — server.js
  %
  % % %  cors()            !’ Checks Origin header, adds CORS response headers
  % % %  express.json()    !’ Parses body !’ req.body = { type, value, unit }
  % % %  router.use(protect) !’
          %
          % % %  Extracts Bearer token from Authorization header
          % % %  jwt.verify(token, JWT_SECRET) !’ decoded = { id: "uuid" }
          % % %  SELECT * FROM users WHERE id = decoded.id !’ req.user
          % % %  next()  !’
               %
               %¼
          usageController.addUsage(req, res)
               %
               % % %  Validate req.body.type, req.body.value
               % % %  INSERT INTO usage_data (user_id, type, value, unit, timestamp)
               %      VALUES ($1, $2, $3, $4, NOW()) RETURNING *
               % % %  If value > threshold !’ INSERT INTO alerts + sendAlertEmail()
               % % %  res.status(201).json({ success:true, data: newRecord })
  %
  %¼
BROWSER receives { success: true, data: { id, type, value, timestamp } }
```

SECTION 17 — 60 VIVA Q&A (MIXED TOPICS)

Q: What is Node.js? NODE

A: A JavaScript runtime built on V8 engine. Enables server-side JS. Uses non-blocking, event-driven I/O — ideal for I/O-heavy apps (APIs, chat, real-time). Single-threaded but handles concurrency via the event loop.

Q: What is the event loop? NODE

A: Node.js runs on a single thread. The event loop continuously checks the call stack (sync code) and the callback/microtask queues (async results). When an async operation (DB query, file read) completes, its callback is queued and executed when the stack is empty.

Q: What is the difference between synchronous and asynchronous code? NODE

A: Synchronous code blocks the thread until it finishes. Async code (callbacks, Promises, async/await) delegates the operation and continues. In Node.js, DB queries are async — the server continues handling other requests while waiting for DB results.

Q: What is npm? NODE

A: Node Package Manager. Manages project dependencies listed in package.json. npm install downloads them to node_modules/. npm start runs the start script. Packages are versioned — ^ means compatible minor updates.

Q: What does dotenv do? NODE

A: The dotenv package reads the .env file and injects values into process.env. Called first in server.js: require("dotenv").config(). This makes DATABASE_URL, JWT_SECRET, etc. available as process.env.DATABASE_URL.

Q: What is Express.js? EXPRESS

A: Minimal web framework for Node.js. Provides routing (app.get, app.post), middleware (app.use), and HTTP utilities. Not opinionated — you choose your DB, auth, structure.

Q: How do you define a route in Express? EXPRESS

A: router.get("/path", middleware, handler). handler = (req, res, next) => {}. req has body, params, query, headers, user. res has json(), status(), send(). Multiple handlers in array = middleware chain.

Q: What is req.params vs req.query vs req.body? EXPRESS

A: req.params: path variables (/alerts/:id !' req.params.id). req.query: URL query string (?severity=red !' req.query.severity). req.body: parsed JSON from POST/PUT request body.

Q: What is app.locals? EXPRESS

A: A property bag for application-level data shared across the app. Used here as app.locals.wsClientCount to track WebSocket connections — accessible in any route handler.

Q: What is PostgreSQL? DB

A: An open-source, ACID-compliant relational database. Uses SQL, supports complex queries, JOINs, transactions, and JSONB columns. More feature-rich than MySQL. Chosen for this project for its reliability and Supabase hosting.

Q: What is ACID in databases? DB

A: Atomicity (transaction fully succeeds or fully fails), Consistency (data always in valid state), Isolation

(concurrent transactions don't interfere), Durability (committed data survives crashes). PostgreSQL is fully ACID-compliant.

Q: What is date_trunc() in PostgreSQL? DB

A: Truncates a timestamp to the specified precision. date_trunc('day', timestamp) removes the time portion, grouping all readings for a day together. Used in dashboard/AI queries to aggregate daily totals.

Q: What is GROUP BY in SQL? DB

A: Collapses multiple rows with the same value in the specified column into a single row. Combined with aggregate functions (SUM, AVG, COUNT) to calculate totals per user/state/day.

Q: What is the difference between WHERE and HAVING? DB

A: WHERE filters rows BEFORE GROUP BY. HAVING filters groups AFTER GROUP BY. Example: HAVING AVG(value) > 100 filters groups by their aggregate, not individual rows.

Q: How are passwords stored securely? AUTH

A: Using bcrypt. The plain password is never stored. bcrypt.genSalt(10) creates a random salt, bcrypt.hash(password, salt) produces an irreversible hash. The hash contains the algorithm, cost factor, salt, and hash — all in one string.

Q: What is Google OAuth? AUTH

A: Allows users to authenticate with their Google account without a password. User consents on Google's page !' Google returns an id_token. Server verifies it with google-auth-library (checks signature, expiry, audience). If valid, the user identity is trusted.

Q: What is the Authorization header format? AUTH

A: "Authorization: Bearer <token>". The "Bearer" scheme indicates a token-based auth. Split by space !' get index 1 for the token. This is the standard for JWT in REST APIs.

Q: What is the purpose of token expiry (30d)? AUTH

A: Limits the window of attack if a token is stolen. After 30 days, the token is invalid and the user must re-login. Shorter expiry (e.g., 15min + refresh tokens) is more secure but adds complexity.

Q: Why implement Z-score instead of using a library? AI

A: The algorithm is straightforward (5 lines of math) and needs no dependencies. Libraries like TensorFlow.js would add 50+ MB for the same result. Custom code is also fully explainable in a viva.

Q: What is the difference between supervised and unsupervised ML? AI

A: Supervised: trained on labelled data (input + known output). Unsupervised: finds patterns without labels. Z-score anomaly detection is unsupervised — no "labelled anomalies" needed, it uses statistical properties of the data itself.

Q: What is overfitting? AI

A: When a model memorizes training data but fails on new data. With exponential smoothing, overfitting is avoided because it uses a simple formula, not a complex fitted model. It generalizes naturally.

Q: What is a confidence interval? AI

A: A range (lower, upper) within which the true value likely falls. Our 10% variance gives an 85% confidence interval — we're 85% confident the actual reading will fall between lower and upper bounds.

Q: What is the difference between WebSocket and SSE (Server-Sent Events)?

A: SSE is one-way (server to client only), simpler to implement, works over HTTP. WebSocket is bidirectional. For a pure broadcast use case like HydroGrid's live feed, SSE would work too. WebSocket was chosen for future bidirectional capability (e.g., user commands to IoT devices).

Q: What happens if a WebSocket client disconnects? WS

A: The ws "close" event fires. wss.clients.size automatically decreases. app.locals.wsClientCount is updated. The disconnected client is removed from the client set — the next broadcast skips it.

Q: What are HTTP status codes? HTTP

A: 1xx: Informational. 2xx: Success (200 OK, 201 Created). 3xx: Redirection. 4xx: Client errors (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found). 5xx: Server errors (500 Internal Server Error).

Q: What is the difference between GET and POST? HTTP

A: GET: retrieve data, no body, idempotent, can be cached. POST: send data in body, create resource, not idempotent. PUT: replace full resource. PATCH: partial update. DELETE: remove resource.

Q: What is idempotent? HTTP

A: An operation is idempotent if calling it multiple times produces the same result as calling it once. GET, PUT, DELETE are idempotent. POST is not (multiple POSTs create multiple records).

Q: What is a rainbow table attack? SEC

A: A precomputed lookup table of password hashes. Attacker steals DB, looks up hash in table !' gets plain password. Defeated by salt — each hash has a unique salt, so identical passwords produce different hashes, making the table useless.

Q: What is privilege escalation? SEC

A: A lower-privileged user gaining admin access. Prevented by: (1) checking role on the SERVER (not trusting client), (2) adminOnly middleware checks req.user.role from DB, not from JWT payload which a client could modify.

Q: What is a CSRF attack? SEC

A: Cross-Site Request Forgery: attacker's website makes requests to our API using victim's browser cookies. JWT in Authorization header (not cookies) prevents this — browsers don't auto-send custom headers cross-site.

Q: What is the MVC pattern? PATTERN

A: Model-View-Controller: Model (data/DB logic), View (frontend/templates), Controller (business logic between model and view). HydroGrid backend follows MVC — routes define paths, controllers handle logic, pg queries are the model layer.

Q: What is async/await? PATTERN

A: Syntactic sugar over Promises. async functions always return a Promise. await pauses execution until the Promise resolves. try/catch handles errors. Makes async code read like synchronous code — avoids .then() chains.

Q: What is destructuring in JavaScript? JS

A: const { name, email, password } = req.body extracts properties. const [first, ...rest] = array extracts elements. Cleaner than req.body.name, req.body.email separately. Used throughout controllers.

Q: What is the spread operator (...)? JS

A: ...array copies array elements: [...arr1, ...arr2]. ...obj copies object properties: { ...existing, newKey: val }. Used in controllers to merge DB results with extra fields without mutation.

Q: What is optional chaining (?.)? JS

A: req.headers.authorization?.startsWith("Bearer") — safely accesses a property without throwing if the value is undefined/null. Returns undefined instead of throwing TypeError.

Q: What is a Promise? JS

A: An object representing the eventual result (or failure) of an async operation. States: pending, fulfilled, rejected. pool.query() returns a Promise. async/await is built on Promises.

Q: How does the admin panel work? PROJECT

A: Admin routes are protected by both protect (JWT check) and adminOnly (role check) middleware. Admin controllers query aggregate stats across ALL users (no user_id filter). Regular user routes always filter by req.user.id.

Q: How are reports generated? PROJECT

A: reportController.js uses pdfkit to create a PDF document server-side. It streams the PDF directly to the response (res) using doc.pipe(res). The frontend triggers a download. CSV export uses manual string building.

Q: What is the /api/usage/map endpoint used for? PROJECT

A: Returns average water/electricity consumption per Indian state, aggregated from all users. This data feeds the India GeoJSON choropleth map on the frontend, colouring states by consumption intensity.

Q: How does badge/gamification system work? PROJECT

A: Badges are stored as JSONB in the users table. On certain events (first reading, 30-day streak, low consumption), the server updates the badges array. The frontend displays them on the Profile page.

Q: What does simulateIoT do? PROJECT

A: Generates realistic random usage records (water 50–800L, electricity 5–35 kWh) for the past 30 days and bulk-inserts them into usage_data. Simulates what a real IoT smart meter would send. Used for demo and testing purposes.

Q: Why use environment variables for secrets? BEST

A: Secrets in code would be committed to git and visible to anyone with repo access. .env file keeps them local, .gitignore prevents committing them. On production (Render/Docker), they are injected at runtime.

Q: Why use try/catch in every controller? BEST

A: DB operations can throw errors (connection lost, constraint violation, timeout). Without try/catch, an unhandled error would crash the server or leave requests hanging. next(error) passes to the global error handler.

Q: What is input validation and where is it done? BEST

A: Checking that required fields exist and are the correct type before processing. Done in controllers (if (! name || !email || !password) return 400). Prevents empty inserts and gives clear error messages to the client.

Q: Why use HTTPS in production? BEST

A: HTTP transmits data in plain text. HTTPS encrypts with TLS — prevents man-in-the-middle attacks where attackers could steal JWT tokens from network traffic. Supabase connection uses SSL. Render/

Vercel provide HTTPS by default.

Q: What is the purpose of app.use(express.json({ limit: "10mb" }))? BEST

A: Parses JSON request bodies. Without this, req.body would be undefined. The 10mb limit prevents denial-of-service attacks via oversized request bodies (e.g., a 1GB JSON payload would exhaust memory without this limit).

Q: What would you improve for production at scale? IMPROVE

A: (1) Rate limiting on /api/auth/login (prevent brute force). (2) Redis cache for dashboard aggregations. (3) JWT in httpOnly cookies (prevent XSS token theft). (4) Database indexes on usage_data(user_id, timestamp). (5) Background job queue (Bull) for email sending. (6) Logging with Winston/Morgan for audit trail.

Q: What is rate limiting and why is it important? IMPROVE

A: Limits how many requests a client can make in a time window. Example: max 5 login attempts per 15 minutes per IP. Implemented with express-rate-limit. Prevents brute-force password attacks and API abuse.

Q: What database indexes would you add? IMPROVE

A: usage_data(user_id) — all queries filter by user_id. usage_data(timestamp) — time-range queries. usage_data(user_id, timestamp) — composite for dashboard queries. users(email) — UNIQUE constraint already creates an index.

SECTION 18 — QUICK REFERENCE CHEAT SHEET

HTTP Methods Summary

Method	Use Case	Body?	Idempotent?	Example
GET	Retrieve data	No	Yes	GET /api/usage
POST	Create resource	Yes	No	POST /api/auth/register
PUT	Full update	Yes	Yes	PUT /api/auth/profile
PATCH	Partial update	Yes	Yes	PATCH /api/alerts/:id
DELETE	Remove resource	No	Yes	DELETE /api/alerts/:id

Key Formulas to Remember

```
Z-score:                z = (x - ;Â’ ò Ã
Anomaly threshold:      |z| > 2.5  !’  flag as anomaly
Exponential smoothing:  S_t = ;  r …÷B ² ƒ  Ò ±) × S_(t-1)   (;  Ò  ã2•
Carbon (electricity):   kWh × 0.82 = kg CO ,
Carbon (water):         litres × 0.001 = kg CO ,
bcrypt work factor:     cost = 10  !’  2^10 = 1024 rounds
JWT expiry:             30 days (30d)
WebSocket push:         every 10,000 ms (10 seconds)
AI cache TTL:           24 hours (86,400,000 ms)
Body size limit:        10 MB
```

All API Endpoints — Final Reference

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login, receive JWT
POST	/api/auth/google	Public	Google OAuth login
GET	/api/auth/profile	JWT	Get own profile
PUT	/api/auth/profile	JWT	Update name/avatar/settings
POST	/api/usage	JWT	Add water/electricity reading
GET	/api/usage	JWT	Get usage history (filterable)
GET	/api/usage/dashboard	JWT	Dashboard aggregated stats
POST	/api/usage/simulate	JWT	Simulate 30 days IoT data
GET	/api/usage/leaderboard	JWT	Efficiency rankings
GET	/api/usage/carbon	JWT	CO2 footprint calculation
GET	/api/usage/tariff-estimate	JWT	State-wise bill estimate
GET	/api/usage/map	JWT	State avg for India map
GET	/api/alerts	JWT	Get alerts (filterable)
PUT	/api/alerts/read-all	JWT	Mark all alerts read
PUT	/api/alerts/:id/read	JWT	Mark specific alert read
DELETE	/api/alerts/:id	JWT	Delete alert
GET	/api/admin/stats	Admin	Platform-wide statistics
GET	/api/admin/users	Admin	All users list
GET	/api/admin/overview	Admin	Monthly overview
GET	/api/admin/dashboard	Admin	Admin dashboard data
GET	/api/ai/detect-anomalies	JWT	Z-score anomaly detection
GET	/api/ai/predict-next-30-days	JWT	Exp. smoothing forecast
GET	/api/ai/recommendations	JWT	AI usage recommendations

GET

/api/ai/device-breakdown

JWT

Device usage breakdown

GET

/api/ai/analytics

JWT

GET

/api/ai/query

JWT

GET

/api/ml/training-data

JWT

POST

/api/ml/train

JWT

Train model on state data

GET

/api/ml/states

JWT

GET

/api/health

Server health check

WS

/ws/live

Real-time metrics stream

HydroGrid Backend Viva Preparation Guide • Generated April 2026 • Node.js / Express / PostgreSQL / JWT / AI / WebSocket

Best of luck with your viva! You've got this.